

Excelente trabajo en la ronda 2: el OpenAPI ahora declara `string/format: uuid` (3 lugares arreglados), agregaste Maven wrapper (`mvnw / mvnw.cmd / .mvn/wrapper/`), `SecurityConfig` + `JwtValidationFilter` están en su lugar, documentaste `bearerAuth` en el OpenAPI, hiciste el publisher de RabbitMQ tolerante a fallos, alineaste el `CreatedOrderResult` con la realidad, y limpiaste el orden de smoke-test de rondas anteriores. **3/3 opcionales cerrados.**

PERO el smoke test contra el auth-service real descubrió **1 blocker** + **2 importantes** antes de cerrar la ronda:

Aspecto	Estado	
OpenAPI drift (integer → uuid)	OK — IDs son UUID en runtime, controllers los aceptan	<code>SecurityConfig</code> + <code>JwtValidationFilter</code> creados
OK — patrón correcto	<code>bearerAuth</code> en OpenAPI	OK
Maven wrapper	OK	Build + tests
OK (13/13)	Boot + <code>\health</code>	OK (1s ready, 200)
Publisher Rabbit tolerante	OK (sin WARN al boot)	<code>CreatedOrderResult</code> schema alignment
OK	Smoke order cleanup	OK
<code>\api/orders</code> con Bearer token válido	ROTO (401)	<code>\api/orders</code> sin token
403 (debería ser 401)	URL del auth-service	hardcodeada localhost:8081 (debería ser configurable)

El filtro autentica contra `http://localhost:8081/auth/validate` con `POST`, pero auth-service expone ese endpoint como **GET** (no POST). Resultado: **incluso con un Bearer token válido, todos los endpoints devuelven 401.**

Confirmado en runtime:

```
GET \api/orders sin Authorization → 403 (Spring Security default, esperado 401)
GET \api/orders con Bearer válido → 401 (porque POST \auth/validate falla)
GET \api/delivery/routes con Bearer → 401 (mismo motivo)
```

Causa raíz

Archivo:

`orders-service/src/main/java/cl/flashdrop/orders/config/JwtValidationFilter.java`, línea 41.

```
authServiceClient.post()
ser .get() // ← debería
```

```
.uri("http://localhost:8081/auth/validate") // ← GET, no POST
.header("Authorization", header)
.retrieve()
.toBodilessEntity();
```

Cómo arreglarlo

Paso 1 — cambiar POST por GET:

```
authServiceClient.get()
    .uri(authServiceUrl + "/auth/validate")
    .header("Authorization", header)
    .retrieve()
    .toBodilessEntity();
```

Tip: en auth-service, el endpoint `/auth/validate` es GET (no POST). Confirmá mirando `auth-service/src/main/java/com/flashdrop/auth/infrastructure/adapters/inbound/rest/AuthController.java`

Paso 2 — verificar con un token real:

Después del fix, el flujo correcto es:

```
\# Sin Authorization → 401
curl -s -w "%{http_code}" http://localhost:8083/api/orders

\# Con Bearer de un login real → 200
TOKEN=$(curl -s -X POST http://localhost:8081/auth/login \
  -H "Content-Type: application/json" \
  -d '{"login":"cliente@demo.cl","password":"123456"}' | jq -r
  '.accessToken')

curl -s -w "%{http_code}" http://localhost:8083/api/orders \
  -H "Authorization: Bearer ${TOKEN}"
```

Esperado: primer curl 401, segundo 200 con datos.

`JwtValidationFilter` tiene `http://localhost:8081/auth/validate` literal en el código. Eso significa que:

- En local funciona (si tenés auth-service en :8081)
- En otros ambientes (staging, prod) **rompe**
- En tests de integración con puertos aleatorios, **rompe**

Cómo arreglarlo

Paso 1 — agregar property en `application-supabase.properties`:

```
auth.service.url=${AUTH_SERVICE_URL:http://localhost:8081}
```

Y también en `.env.example`:

```
AUTH\_SERVICE\_URL=http://localhost:8081
```

Paso 2 — inyectar la URL en el filter:

```
\@Component
public class JwtValidationFilter extends OncePerRequestFilter {

    private static final Logger logger \=
LoggerFactory.getLogger(JwtValidationFilter.class);

    private final RestClient authServiceClient;
    private final String authServiceUrl;

    public JwtValidationFilter(
        \@Value("\${auth.service.url}") String authServiceUrl) {
        this.authServiceUrl \= authServiceUrl;
        this.authServiceClient \= RestClient.create();
    }

    \@Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
FilterChain chain)
        throws ServletException, IOException {

        String header \= req.getHeader("Authorization");

        if (header != null && header.startsWith("Bearer ")) {
            String token \= header.substring(7);

            try {
                ResponseEntity<Void> validation \= authServiceClient.get()
                    .uri(authServiceUrl \+ "/auth/validate")
                    .header("Authorization", header)
                    .retrieve()
                    .toBodilessEntity();

                if (validation.getStatusCode().is2xxSuccessful()) {
                    UsernamePasswordAuthenticationToken authentication \=
                        new UsernamePasswordAuthenticationToken(token, null, new
ArrayList<>());
                    SecurityContextHolder.getContext().setAuthentication(authentication);
                }

            } catch (RestClientException e) {
                logger.warn("Auth service validation failed: {}", e.getMessage());
                res.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Invalid token");
                return;
            }
        }

        chain.doFilter(req, res);
    }
}
```

```
}  
}
```

Hoy con `JwtValidationFilter` aplicado, las requests sin token devuelven **403** (Spring Security default). La convención HTTP es:

- **401 Unauthorized:** el cliente **no se autenticó** (no envió credenciales o son inválidas)
- **403 Forbidden:** el cliente se autenticó pero **no tiene permisos**

Diferencia importante: muchos clientes y proxies (incluido el gateway) manejan estos códigos distinto.

Cómo arreglarlo

En `SecurityConfig.java`:

```
@Bean  
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {  
    http  
        .csrf(csrf -> csrf.disable())  
        .sessionManagement(session ->  
            session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))  
        .authorizeHttpRequests(authz -> authz  
            .requestMatchers("/api/orders/**", "/api/delivery/  
/**").authenticated()  
            .anyRequest().permitAll()  
        )  
        .exceptionHandling(ex -> ex  
            .authenticationEntryPoint((req, res, e) ->  
                res.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Unauthorized"))  
            .accessDeniedHandler((req, res, e) ->  
                res.sendError(HttpServletResponse.SC_FORBIDDEN, "Forbidden"))  
        )  
        .addFilterBefore(jwtValidationFilter,  
            UsernamePasswordAuthenticationFilter.class);  
  
    return http.build();  
}
```

Después de aplicar esto:

Situación	Esperado	
Sin Authorization en <code>/api/orders/**</code>	401	Con Bearer inválido en <code>/api/orders/**</code>
401	Con Bearer válido pero rol insuficiente	403

Paso 1 — Fix `JwtValidationFilter` (POST → GET)

Editá `orders-service/src/main/java/cl/flashdrop/orders/config/JwtValidationFilter.java` :

- Cambiá `.post()` por `.get()` en línea 41
- Externalizá la URL del auth-service vía `@Value("${auth.service.url}")` (Importante 1)
- Agregá `auth.service.url` a `application-supabase.properties` y a `.env.example`

```
./mvnw clean package \-DskipTests=false
\# BUILD SUCCESS
git add orders-service/src/main/java/cl/flashdrop/orders/config/
JwtValidationFilter.java
git add orders-service/src/main/resources/application-supabase.properties
git add orders-service/.env.example
git commit \-m \"fix(orders): JwtValidationFilter calls GET \auth/validate (was POST
401); externalize URL via property\"
```

Paso 2 — Homogeneizar 401 vs 403

Editá `SecurityConfig.java` y agregá el `exceptionHandling(...)` del Importante 2.

```
git add orders-service/src/main/java/cl/flashdrop/orders/config/SecurityConfig.java
git commit \-m \"fix(orders): return 401 (not 403) when Authorization is missing/
invalid\"
```

Paso 3 — Verificar end-to-end con token real

Levantá auth-service y orders-service en paralelo (el puerto 8081 y 8083 no chocan). Después:

```
\# Boot auth-service
cd juniors/junior-1-auth/auth-service
./gradlew bootRun \-Dspring.profiles.active=supabase \-Dorg.gradle.java.home=/home/
pelle/jdk/jdk-21.0.2 \--no-daemon &
AUTH_PID=\$!

\# Wait for ready
for i in \$(seq 1 90); do
  if curl \-sf http://localhost:8081/actuator/health \> \dev/null 2\>&1; then break;
fi
  sleep 1
done

\# Boot orders-service
cd juniors/junior-2-orders/orders-service
./mvnw spring-boot:run \-Dspring-boot.run.profiles=supabase &
ORDERS_PID=\$!

for i in \$(seq 1 90); do
  if curl \-sf http://localhost:8083/health \> \dev/null 2\>&1; then break; fi
  sleep 1
done
```

```

\# Sin auth → 401
curl \-s \-w "\nHTTP\=%{http\_code}\n" http://localhost:8083/api/orders

\# Login en auth-service, obtener token
TOKEN\=$(curl \-s \-X POST http://localhost:8081/auth/login \-H "Content-Type: application/json" \-d '{"login\":"cliente@demo.cl","password\":"123456"}' \- | grep \-oP '"accessToken\":"[^"]*" | cut \-d\'\' \-f4)

\# Con Bearer → 200
curl \-s \-w "\nHTTP\=%{http\_code}\n" http://localhost:8083/api/orders \-H "Authorization: Bearer \${TOKEN}"

```

Esperado:

- Primera llamada: **HTTP 401**
- Segunda llamada: **HTTP 200** con lista de orders

Paso 4 — Push al PR #1

```

git push origin feat/orders-supabase-rest
\# Comentario en PR \#1: "Round-3 fix: JwtValidationFilter calls GET \-auth/validate (was POST → 401 always). URL externalized. 401/403 conventions aligned."

```

(Receta del feedback de ronda 2, mantenida por si la perdiste.)

1. Crear `.env` desde cero en `orders-service/.env`

```

cd juniors/junior-2-orders/orders-service

cat \> .env \<\<'EOF'
\# \--- Supabase REST (perfil supabase) \---
SUPABASE\_URL\=http://supabasekong-wymwq8rktid7ov678oe4va90.76.13.169.150.sslip.io
SUPABASE\_SERVICE\_ROLE\_KEY\=\<pedirle al líder del equipo – NUNCA versionar\>
SPRING\_PROFILES\_ACTIVE\=supabase

\# \--- Auth-service URL (nueva en R3) \---
AUTH\_SERVICE\_URL\=http://localhost:8081

\# \--- Business config (defaults) \---
DELIVERY\_FEE\=2500
EOF

```

Tip R3: el `.env.example` también debe tener `AUTH\_SERVICE\_URL` para que cualquier dev nuevo sepa qué poner.

2. Setear JAVA_HOME

```
export JAVA_HOME=/path/a/tu/jdk-21
export PATH=\ "$JAVA_HOME/bin:\$PATH"
```

3. Build + boot

```
chmod +x mvnw
./mvnw clean package -DskipTests=false
./mvnw spring-boot:run -Dspring-boot.run.profiles=supabase
```

4. Probar

```
curl http://localhost:8083/health
\# Esperado: {"service":"orders-service","status":"ok"}

\# Sin auth – esperado 401 (después del fix de R3)
curl -s -w "\nHTTP=%{http_code}\n" http://localhost:8083/api/orders
```

-
- **Lo que hiciste bien en ronda 2:** OpenAPI migrado a UUID, Maven wrapper agregado, publisher tolerante, schema alignment, smoke-test cleanup, security infra armada con SecurityConfig + JwtValidationFilter.
 - **Lo que falta:** el filter llama al endpoint equivocado (POST en vez de GET), tiene la URL hardcodeada, y devuelve 403 en vez de 401. Tres fixes chicos, todos en el mismo módulo de auth.
 - **Próxima ronda esperada:** después del GET/POST fix + URL externalization + 401/403 alignment, la auth queda production-ready para orders.

Cuando termines, subí los cambios y avisame para volver a probar end-to-end con auth-service + orders-service en paralelo.

- **Spring Security exception handling:** <https://www.baeldung.com/spring-security-exception-handling>
- **RestClient @Value config:** <https://docs.spring.io/spring-framework/reference/integration/rest-clients.html#rest-http-interface>
- **HTTP 401 vs 403:** <https://stackoverflow.com/questions/3297048/403-forbidden-vs-401-unauthorized-responses>
- **auth-service controllers** (referencia para el método HTTP correcto de `/auth/validate`):
`juniors/junior-1-auth/auth-service/src/main/java/com/flashdrop/auth/infrastructure/adaptor/inbound/rest/AuthController.java`